M03 - Advanced SQL + PostgreSQL / Database Thinking

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

How to read this book

Pretend a patient instructor is sitting beside you. Every new term is explained before it is used. Type the example, inspect the result, answer the STOP question, and only then move to the next lesson.

Contents

Official PostgreSQL current documentation | Official PostgreSQL Windows download

DE03-01 - INNER JOIN and LEFT JOIN

What you will be able to do

Join related tables without accidentally losing rows or changing the question.

Why this matters in real work

A real company rarely stores everything in one giant table. Orders may live in one table and customer names or tiers in another. A JOIN lets you combine them at query time.

Picture it this way

Imagine two school registers. One register has exam attempts and student IDs. Another has student IDs and names. A JOIN matches the same ID across the two registers.

Start from zero

- **INNER JOIN** keeps only rows that find a match on both sides.
- **LEFT JOIN** keeps every row from the table written on the left, even if the right side has no match.
- The **ON** condition defines what a match means. In our retail lab, orders and customers match on customer_id.
- Before joining, say the input grain aloud. sales_orders is one row per order. customers is one row per customer.
- After joining a one-to-one customer table, the intended output can still be one row per order.

Teacher demonstration

```
SELECT
    o.order_id,
    o.customer_id,
    c.customer_name,
    c.tier
FROM sales_orders AS o
LEFT JOIN customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_id
LIMIT 5;
```

Read the SQL like English

- **o** and **c** are short aliases. They make long queries easier to read.
- The query starts from sales_orders, so a LEFT JOIN promises to preserve those order rows.
- c.customer_name comes from customers only when the customer_id match succeeds.
- In the supplied clean lab data, all 30 orders have a matching customer, so INNER and LEFT both return 30 rows. The difference becomes visible when a left-side row has no match.

Expected check / validation anchor

check	expected
sales_orders rows	30
INNER JOIN rows	30
LEFT JOIN rows	30

Common beginner traps

- Choosing INNER because it is familiar even when the business asks for all left-side rows.
- Joining on a descriptive name instead of the intended key.
- Adding a WHERE condition on the right table after a LEFT JOIN and accidentally removing unmatched rows.

Now you do a similar task

List Completed West orders with order_id, customer_name, tier and net_sales. Before running it, write: input grain = one row per order; intended output grain = one row per matching Completed West order.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

A report must show every order, even if customer details are missing. Which join should you start with, and why?

Do not turn the page until you can answer in your own words.

DE03-02 - Cardinality, row multiplication and join validation

What you will be able to do

Recognize when a join legitimately multiplies rows and prove whether the output grain changed.

Why this matters in real work

A query can return believable totals while being badly duplicated. Hidden one-to-many relationships are one of the most expensive SQL mistakes in analytics and data engineering.

Picture it this way

One customer can have several phone numbers. If one order is joined to all of that customer's phone numbers, that single order can appear multiple times. Nothing is 'random'; the relationship caused the multiplication.

Start from zero

- **Cardinality** describes how many rows on one side can relate to rows on the other side: one-to-one, one-to-many, or many-to-many.
- customer_contacts has multiple rows for some customers because one customer can have Primary, Billing, and Warehouse contacts.
- Joining 30 orders to customer_contacts produces **35 rows**, while the distinct order_id count remains **30**.
- That means the join output is no longer guaranteed to be one row per order. Some orders are now one row per order-contact combination.
- DISTINCT can hide symptoms. It does not explain whether multiplication is legitimate.

Teacher demonstration

```
SELECT
    COUNT(*) AS joined_rows,
    COUNT(DISTINCT o.order_id) AS distinct_orders
FROM sales_orders AS o
LEFT JOIN customer_contacts AS cc
    ON o.customer_id = cc.customer_id;
```

Read the SQL like English

- COUNT(*) counts physical result rows after the join.
- COUNT(DISTINCT o.order_id) counts unique orders inside those rows.
- If those numbers differ, one or more orders appeared more than once.
- In the supplied data, joined_rows = 35 and distinct_orders = 30.

Expected check / validation anchor

metric	expected
joined_rows	35
distinct_orders	30
largest multiplication	O1003 appears 3 times

Common beginner traps

- Adding DISTINCT before investigating why rows multiplied.
- Comparing only total row counts and forgetting distinct business keys.
- Assuming a foreign key automatically means one-to-one.

Now you do a similar task

Find every order_id that appears more than once after joining sales_orders to customer_contacts. For each repeated order, inspect the related contact_type values and explain why the multiplication occurred.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

If a join returns 35 rows but only 30 distinct order IDs, what changed: the stored sales_orders table, or only the result grain?

Do not turn the page until you can answer in your own words.

DE03-03 - UNION and UNION ALL

What you will be able to do

Stack compatible result sets and choose deliberately between UNION ALL and UNION.

Why this matters in real work

Data engineers often combine events from different sources into one stream: orders, support tickets, refunds, log events, or files from different months.

Picture it this way

Imagine placing two piles of index cards into one box. UNION ALL keeps every card. UNION checks for duplicate cards and removes duplicates according to the selected columns.

Start from zero

- **UNION ALL** appends every row and does not perform duplicate elimination.
- **UNION** also appends rows but removes duplicate result rows, which can erase legitimate repeated events.
- Both SELECT statements need the same number of columns in compatible positions.
- Use a source/event_type label so you can still tell where each row came from.
- In our lab there are 27 Completed orders and 10 tickets, so a UNION ALL event stream should contain 37 rows.

Teacher demonstration

```
SELECT
    order_id AS event_id,
    customer_id,
    order_date AS event_date,
    'Order' AS event_type
FROM sales_orders
WHERE status = 'Completed'
UNION ALL
SELECT
    ticket_id,
    customer_id,
    DATE(created_at),
    'Ticket'
FROM support_tickets
ORDER BY event_date, event_id;
```

Read the SQL like English

- The first SELECT produces four columns.
- The second SELECT also produces four columns in the same semantic order.
- The literal labels 'Order' and 'Ticket' preserve source meaning.
- UNION ALL is appropriate because two events that happen to have equal displayed values can still be legitimate separate events.

Expected check / validation anchor

source	rows
Completed orders	27
Support tickets	10

source	rows
Combined UNION ALL stream	37

Common beginner traps

- Using UNION automatically because it sounds safer.
- Stacking columns that have the same data type but different meaning.
- Forgetting a source label and later being unable to identify event origin.

Now you do a similar task

Build the activity stream yourself, then validate it with a second query that counts rows by event_type. Your validation should reconcile to 27 Order rows and 10 Ticket rows.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

Why can UNION be dangerous when duplicate-looking rows may represent real separate events?

Do not turn the page until you can answer in your own words.

DE03-04 - String functions for cleaning and standardization

What you will be able to do

Standardize messy text in query output while preserving the raw source.

Why this matters in real work

Names and labels often arrive with mixed case or accidental spaces. Standardizing in a transformation query makes matching and reporting more reliable without destroying the original evidence.

Picture it this way

You are not repainting the original signboard. You are creating a clean display copy for the report.

Start from zero

- **TRIM** removes leading and trailing spaces.
- **UPPER** and **LOWER** standardize case in the returned value.
- **CONCAT** combines pieces into a readable label.
- **LIKE** performs simple pattern matching. Percent signs act as wildcards.
- A SELECT expression changes the result, not the stored source value.

Teacher demonstration

```
SELECT
    customer_id,
    customer_name,
    UPPER(TRIM(customer_name)) AS name_clean,
    CONCAT(customer_name, ' - ', tier) AS customer_label
FROM customers
WHERE customer_name LIKE '%Retail%';
```

Read the SQL like English

- The raw customer_name is still returned so you can compare raw vs cleaned.
- UPPER(TRIM(...)) nests two functions: trim first, then uppercase.
- LIKE '%Retail%' means Retail may appear anywhere inside the name.

Expected check / validation anchor

customer_id	customer_name
C003	Ravi Retail
C007	Om Retail
C015	Ishita Retail
C020	Saanvi Retail
C026	Sara Retail

Common beginner traps

- Cleaning the result and assuming the source table was repaired.
- Dropping the original value too early, making auditing harder.
- Using a pattern filter without checking whether case sensitivity differs by database/collation.

Now you do a similar task

Return every customer with original name, lowercase name, and a label in the form customer_name | home_region | tier. Keep the raw name beside the cleaned version.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

If UPPER(customer_name) looks clean in the result grid, did PostgreSQL/MySQL permanently change customer_name in the table?

Do not turn the page until you can answer in your own words.

DE03-05 - Dates, timestamps and time analysis

What you will be able to do

Calculate elapsed time correctly and state which business timestamps define the interval.

Why this matters in real work

Time metrics are easy to calculate and easy to misinterpret. 'Return delay' could mean from order date, shipment date, delivery date, or refund approval date.

Picture it this way

A stopwatch is useful only if everyone agrees when you pressed Start and Stop.

Start from zero

- A **DATE** stores a calendar date. A timestamp/datetime stores date plus time of day.
- MySQL DATEDIFF(end_date, start_date) returns a day difference based on dates.
- The business meaning comes from the columns you choose, not from the DATEDIFF function itself.
- Time zones matter when timestamps cross midnight in different locations.
- For this lab, days_to_return means return_date minus order_date.

Teacher demonstration

```
SELECT
    r.return_id,
    r.order_id,
    o.order_date,
    r.return_date,
    DATEDIFF(r.return_date, o.order_date) AS days_to_return
FROM returns AS r
JOIN sales_orders AS o
    ON r.order_id = o.order_id
ORDER BY days_to_return DESC, r.return_id;
```

Read the SQL like English

- Returns must join to orders because return_date and order_date live in different tables.
- The longest delay in this small dataset is 4 days.
- O1007 and O1015 both have a 4-day delay.

Expected check / validation anchor

order_id	days_to_return
O1007	4
O1015	4
O1002	3
O1023	3
O1026	3

Common beginner traps

- Subtracting two date columns without knowing how your database treats date arithmetic.
- Calling the metric 'refund processing time' when it actually uses order_date and return_date.
- Ignoring time zone conversion for timestamp-based production metrics.

Now you do a similar task

Find the maximum and average days_to_return. Write one sentence defining exactly what your interval starts from and ends at.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

Why is the SQL expression alone not enough to define a trustworthy business time metric?

Do not turn the page until you can answer in your own words.

DE03-06 - Subqueries and nested logic

What you will be able to do

Solve a layered question with a subquery and validate the inner question separately.

Why this matters in real work

Many business questions have a natural two-step structure: first calculate a benchmark, then compare rows with that benchmark.

Picture it this way

First ask the class average. Then ask which students scored above it. The second question cannot be answered correctly until the first number is trusted.

Start from zero

- A **scalar subquery** returns one value, such as an average.
- The inner query should be runnable and understandable by itself.
- The outer query uses the inner result as an input.
- A **correlated subquery** depends on values from the outer row; we only need awareness here.
- Readable SQL is more valuable than clever nesting.

Teacher demonstration

```
SELECT
    order_id,
    qty * unit_price * (1 - discount_pct) AS net_sales
FROM sales_orders
WHERE status = 'Completed'
  AND qty * unit_price * (1 - discount_pct) > (
      SELECT AVG(qty * unit_price * (1 - discount_pct))
      FROM sales_orders
      WHERE status = 'Completed'
  )
ORDER BY net_sales DESC;
```

Read the SQL like English

- Run the inner AVG query first. In this dataset the Completed-order average net_sales is about 1592.02.
- Then run the full query. There are 13 Completed orders above that average.
- The same status definition appears in inner and outer logic so the benchmark and compared rows refer to the same population.

Expected check / validation anchor

metric	expected
Completed average net_sales	1592.02 approx
Rows above average	13
Highest row	O1020 = 3060.00

Common beginner traps

- Comparing Completed rows against an average that includes Cancelled/Pending rows.
- Never running the inner query alone.
- Repeating a long expression inconsistently in multiple places.

Now you do a similar task

Return Completed orders above the Completed-order average gross_sales. Save the inner average query separately, then the outer query, then a count of returned rows.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

If your outer query is wrong, which smaller piece should you prove first?

Do not turn the page until you can answer in your own words.

DE03-07 - CTEs and readable multi-step SQL

What you will be able to do

Rewrite multi-step SQL with named CTEs so each transformation step and grain is visible.

Why this matters in real work

Data engineering queries often grow beyond one screen. CTEs help you name intermediate datasets and reason about them one step at a time.

Picture it this way

Instead of solving a long math problem in one line, write Step A, Step B, then the final answer. Each step has a name and can be inspected.

Start from zero

- **WITH** starts one or more Common Table Expressions (CTEs).
- A CTE is a named query result used by the statement that follows.
- Give each CTE a job and a clear grain. For example, order_sales = one row per order.
- CTEs improve readability and testability; they are not a promise of faster execution.
- During development, temporarily SELECT * FROM one CTE to inspect it.

Teacher demonstration

```
WITH order_sales AS (
    SELECT
        order_id,
        region,
        status,
        qty * unit_price * (1 - discount_pct) AS net_sales
    FROM sales_orders
),
completed_avg AS (
    SELECT AVG(net_sales) AS avg_net_sales
    FROM order_sales
    WHERE status = 'Completed'
)
SELECT o.order_id, o.region, o.net_sales
FROM order_sales AS o
CROSS JOIN completed_avg AS a
WHERE o.status = 'Completed'
  AND o.net_sales > a.avg_net_sales
ORDER BY o.net_sales DESC;
```

Read the SQL like English

- order_sales has one row per order and calculates net_sales once.
- completed_avg has one row total: the benchmark value.
- The final SELECT combines those pieces and applies the comparison.

Expected check / validation anchor

CTE	grain
order_sales	one row per order
completed_avg	one row total

CTE	grain
final output	one row per above-average Completed order

Common beginner traps

- Using ten CTEs when two clear steps are enough.
- Naming CTEs step1/step2 instead of describing their meaning.
- Changing grain inside a CTE without documenting it.

Now you do a similar task

Rewrite your DE03-06 gross-sales solution using two named CTEs. Put a SQL comment above each CTE stating its grain.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

What is the grain of a CTE, and why should you write it down?

Do not turn the page until you can answer in your own words.

DE03-08 - Window functions and PARTITION BY

What you will be able to do

Use a window function to calculate across related rows without collapsing order-level detail.

Why this matters in real work

GROUP BY is excellent for summaries, but sometimes you need the summary beside every detailed row - for example each order beside its regional average.

Picture it this way

A teacher writes the class average beside every student's individual score. The student rows remain; the summary is added beside them.

Start from zero

- A window function calculates across a set of related rows while usually keeping the original row count.
- **OVER (...)** defines the window.
- **PARTITION BY region** creates a separate calculation group for each region.
- GROUP BY one row per region would collapse detail; AVG(...) OVER(PARTITION BY region) does not.
- Always prove row-count preservation when that is part of the requirement.

Teacher demonstration

```
SELECT
    order_id,
    region,
    qty * unit_price * (1 - discount_pct) AS net_sales,
    AVG(qty * unit_price * (1 - discount_pct))
        OVER (PARTITION BY region) AS region_avg_sales
FROM sales_orders
WHERE status = 'Completed'
ORDER BY region, order_id;
```

Read the SQL like English

- WHERE first limits rows to Completed orders.
- The window average is calculated separately inside each region.
- The result still has 27 rows because there are 27 Completed orders.

Expected check / validation anchor

region	completed_rows	region_avg_net_sales
East	8	1549.06
North	5	1752.90
South	7	1613.57
West	7	1504.64

Common beginner traps

- Using GROUP BY and then wondering where individual order IDs went.
- Forgetting that WHERE changes which rows participate in the window.
- Confusing final ORDER BY with ORDER BY inside OVER for sequence-sensitive windows.

Now you do a similar task

Add region_avg_sales beside each Completed order and add a CASE flag 'Above region avg' / 'At or below'. Prove the final row count is still 27.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

Why does a window average preserve order rows while GROUP BY region does not?

Do not turn the page until you can answer in your own words.

DE03-09 - ROW_NUMBER, ranking and LAG/LEAD

What you will be able to do

Solve latest-row, ranking, and previous-event questions with ordered window functions.

Why this matters in real work

These patterns appear constantly in event data: latest status per customer, top products per region, previous login, previous order, and change from the last measurement.

Picture it this way

For each customer, arrange their event cards in time order. ROW_NUMBER labels the positions; LAG lets the current card look one card backward.

Start from zero

- **ROW_NUMBER()** gives 1,2,3... inside a partition according to the specified order.
- For 'latest', order descending by timestamp so the newest row gets row_number = 1.
- Use a deterministic tie-breaker such as ticket_id when timestamps could tie.
- **RANK** leaves gaps after ties; **DENSE_RANK** does not. ROW_NUMBER never ties.
- **LAG** returns a value from a previous ordered row inside the same partition.

Teacher demonstration

```
WITH ranked_tickets AS (
    SELECT
        ticket_id,
        customer_id,
        created_at,
        status,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY created_at DESC, ticket_id DESC
        ) AS rn
    FROM support_tickets
)
SELECT ticket_id, customer_id, created_at, status
FROM ranked_tickets
WHERE rn = 1
ORDER BY customer_id;
```

Read the SQL like English

- PARTITION BY customer_id restarts numbering for every customer.
- Newest created_at gets rn = 1.
- ticket_id is the tie-breaker if two tickets have the same timestamp.
- The dataset produces one latest ticket for each of 5 customers with tickets.

Expected check / validation anchor

customer_id	latest_ticket
C001	T002
C003	T005

customer_id	latest_ticket
C005	T007
C010	T009
C020	T010

Common beginner traps

- Using MAX(created_at) and then selecting non-aggregated columns without a safe way to return the matching row.
- Omitting a tie-breaker when 'latest' must be deterministic.
- Partitioning by region when the sequence question is actually per customer.

Now you do a similar task

A) Return the latest ticket per customer. B) On sales_orders, use LAG(order_date) over each customer_id and calculate or display the previous order date.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

For 'latest ticket per customer', what belongs in PARTITION BY and what belongs in ORDER BY?

Do not turn the page until you can answer in your own words.

DE03-10 - Advanced aggregation and multi-grain reasoning

What you will be able to do

Reason explicitly across multiple grains so ratios use compatible numerators and denominators.

Why this matters in real work

A large share of 'SQL is wrong but looks right' problems come from mixing grains. This is a core data-engineering habit, not a trick.

Picture it this way

Suppose a school has 100 students and 140 exam attempts. 'Failed attempts / students' is not a failure rate unless that is the definition you intended. The numerator counts attempts; the denominator counts people. They are different grains.

Start from zero

- **Grain** means what one row represents at a given step.
- Our source sales_orders is one row per order. returns is one row per return event in this lab.
- The final question is: **for each region, what percentage of distinct orders were returned?**
- The denominator must therefore be distinct orders per region. The numerator must be distinct returned orders per the same region.
- Joining orders to another one-to-many table before counting can multiply physical rows, so COUNT(*) may become unsafe.
- COUNT(DISTINCT order_id) is not magic; it is correct here because the business entity in both numerator and denominator is an order.

Teacher demonstration

```
SELECT
    o.region,
    COUNT(DISTINCT o.order_id) AS orders_count,
    COUNT(DISTINCT r.order_id) AS returned_orders,
    ROUND(
        100.0 * COUNT(DISTINCT r.order_id)
        / NULLIF(COUNT(DISTINCT o.order_id), 0),
        2
    ) AS return_rate_pct
FROM sales_orders AS o
LEFT JOIN returns AS r
    ON o.order_id = r.order_id
GROUP BY o.region
ORDER BY o.region;
```

Read the SQL like English

- **Step 1 grain:** sales_orders = one row per order.
- **Step 2 after LEFT JOIN:** one row per order-return match. In this toy data there is at most one return row per returned order, but production data may have more.
- **Final grain:** one row per region.
- **Denominator:** distinct order IDs in the region.

- **Numerator:** distinct order IDs that have a return match.
- Expected all-order rates: East 0/8 = 0%; North 2/7 = 28.57%; South 0/7 = 0%; West 3/8 = 37.50%.

Expected check / validation anchor

region	orders_count	returned_orders	return_rate_pct
East	8	0	0.00
North	7	2	28.57
South	7	0	0.00
West	8	3	37.50

Common beginner traps

- Dividing return event rows by customer rows.
- Using COUNT(*) after a one-to-many join without proving that each order still appears once.
- Changing the denominator to Completed orders but still describing the result as 'all-order return rate'.
- Using DISTINCT as a patch without stating which entity is being counted.

Now you do a similar task

Recalculate the metric for Completed orders only. Your final grain must still be one row per region. Expected denominators become East 8, North 5, South 7, West 7. Explain why the North/West rates increase.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

If your numerator counts returned orders but your denominator counts joined rows after a one-to-many contact join, are those two numbers at compatible grains? Explain.

Do not turn the page until you can answer in your own words.

DE03-11 - PostgreSQL setup, schemas and dialect awareness

What you will be able to do

Move from MySQL to PostgreSQL without treating PostgreSQL as 'the same program with a different logo'.

Why this matters in real work

Data engineers frequently work with PostgreSQL. Most SQL concepts transfer, but database objects, administration, functions, and dialect details differ.

Picture it this way

SQL is like a language family. MySQL and PostgreSQL share many words, but each has its own accent, grammar choices, tools, and house rules.

Start from zero

- A PostgreSQL server can contain multiple **databases**. Inside a database, **schemas** organize named objects such as tables and views.
- A useful beginner picture is: PostgreSQL server -> database -> schema -> table.
- A schema is similar to a named folder/namespace inside one database. Two schemas can each contain a table called orders.
- PostgreSQL normally includes a schema named **public**. An unqualified table name such as orders is resolved using the **search_path**.
- **SHOW search_path;** displays that lookup path. The first matching object name in the path is used.
- A qualified name such as **training.orders** tells PostgreSQL exactly which schema to use.
- PostgreSQL supports LIMIT, uses single quotes for string values, and double quotes only when you intentionally need quoted identifiers.
- Do not copy MySQL-specific setup commands such as USE database_name into PostgreSQL and assume they work unchanged.

Teacher demonstration

```
-- Run inside a PostgreSQL database in pgAdmin Query Tool
SHOW search_path;
CREATE SCHEMA training;
CREATE TABLE training.orders (
    order_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    amount NUMERIC(10,2) CHECK (amount >= 0)
);
INSERT INTO training.orders VALUES
(1, 'Aman', 500.00),
(2, 'Riya', 800.00),
(3, 'Neha', 350.00);
SELECT *
FROM training.orders
ORDER BY order_id;
```

Read the SQL like English

- CREATE SCHEMA training creates a namespace named training inside the database you are connected to.

- training.orders means table orders inside schema training.
- INTEGER, TEXT, and NUMERIC are column data types.
- PRIMARY KEY says order_id is the row identity.
- NOT NULL says customer_name cannot be missing.
- CHECK (amount >= 0) rejects negative amounts.
- The supplied M03 PostgreSQL lab script contains this example and additional safe exercises.

Expected check / validation anchor

order_id	customer_name	amount
1	Aman	500.00
2	Riya	800.00
3	Neha	350.00

Common beginner traps

- Typing USE stage03_retail_lab; in PostgreSQL because you learned it in MySQL.
- Thinking schema and database are the same level.
- Quoting normal lowercase identifiers with double quotes everywhere, which can create avoidable case-sensitivity surprises.
- Running setup in the wrong database because the pgAdmin Query Tool was connected elsewhere.

Now you do a similar task

Create a schema named practice and a table practice.students(student_id, student_name, score). Insert 3 rows, query them, then run SHOW search_path. Do not change search_path yet; simply explain why practice.students works even if practice is not first in the path.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

In the name training.orders, which part is the schema and which part is the table? Why can that be safer than writing only orders?

Do not turn the page until you can answer in your own words.

DE03-12 - Constraints and transactions

What you will be able to do

Use database constraints to prevent invalid states and transactions to group related changes safely.

Why this matters in real work

A data pipeline should not depend on every application remembering every rule. The database can enforce important invariants close to the data.

Picture it this way

Constraints are entry rules at a gate. A transaction is a sealed set of actions: either the whole set is accepted, or the whole set is undone.

Start from zero

- **PRIMARY KEY** protects row identity and uniqueness.
- **FOREIGN KEY** protects references between related tables.
- **NOT NULL** prevents a missing value where the business requires one.
- **CHECK** validates a condition such as amount >= 0.
- A transaction starts with BEGIN, is made permanent with COMMIT, or discarded with ROLLBACK.
- Atomicity matters when multiple writes represent one business action.

Teacher demonstration

```
BEGIN;
UPDATE inventory
SET qty = qty - 1
WHERE sku = 'KB-01';
INSERT INTO shipment_log(sku, shipped_qty)
VALUES ('KB-01', 1);
-- If validation fails:
ROLLBACK;
-- If everything is valid:
-- COMMIT;
```

Read the SQL like English

- The two statements represent one business action: reduce stock and record the shipment.
- If the second step fails, committing only the first step would leave inconsistent state.
- ROLLBACK returns the transaction's changes to the starting state if they were not committed.

Expected check / validation anchor

rule	example
row identity	PRIMARY KEY
required value	NOT NULL
valid range	CHECK
valid parent reference	FOREIGN KEY

Common beginner traps

- Using constraints only as documentation instead of allowing the database to enforce them.
- Running destructive UPDATE/DELETE statements outside a transaction during experimentation.
- Assuming ROLLBACK can undo a transaction that has already been committed.

Now you do a similar task

Design a PostgreSQL orders table with: unique order identity, required customer_id, amount that cannot be negative, and status restricted to Pending/Completed/Cancelled. Then describe one two-step business update that should be wrapped in a transaction.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

What problem does a CHECK constraint prevent that a comment in application code cannot reliably prevent?

Do not turn the page until you can answer in your own words.

DE03-13 - Indexes and EXPLAIN basics

What you will be able to do

Choose an index because of an actual access pattern and use EXPLAIN evidence instead of guessing.

Why this matters in real work

Indexes can speed reads but cost storage and write work. Data engineers need evidence, not the superstition that 'more indexes = faster database'.

Picture it this way

A book index helps you jump to a topic instead of reading every page. But maintaining an index takes extra space and work whenever the book changes.

Start from zero

- A database index is a separate structure that helps the planner find rows efficiently for certain search/join/order patterns.
- An index is useful only when it matches real query patterns and the planner estimates it is worthwhile.
- **EXPLAIN** shows the chosen plan without executing the query in PostgreSQL.
- **EXPLAIN ANALYZE** executes the query and reports actual timing/row information; use it carefully on modifying statements or expensive production queries.
- Small toy tables may still use a sequential scan even when an index exists. That does not automatically mean the index is broken.
- Measure before/after on representative data.

Teacher demonstration

```
EXPLAIN
SELECT *
FROM sales_orders
WHERE customer_id = 'C010';
CREATE INDEX idx_sales_orders_customer_id
    ON sales_orders(customer_id);
EXPLAIN
SELECT *
FROM sales_orders
WHERE customer_id = 'C010';
```

Read the SQL like English

- The query pattern filters on customer_id, so customer_id is a plausible index key.
- Record the plan before creating the index.
- Create the index.
- Record the plan after. On a tiny table the planner may still prefer a sequential scan.
- Your conclusion must be about evidence: plan shape, estimated rows, actual timing on realistic data - not about what you hoped would happen.

Expected check / validation anchor

evidence	what to record
before	plan node / estimated cost / row estimate
after	same fields after index
conclusion	state whether evidence supports improvement; do not force a claim

Common beginner traps

- Creating an index on every column.
- Saying 'index made it faster' without plan/runtime evidence.
- Using EXPLAIN ANALYZE on a destructive statement without understanding that it actually executes.

Now you do a similar task

For a frequent query filtering customer_id and ordering by order_date, propose one index. Write the query pattern first, then the index, then exactly what evidence you would compare before/after.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

Why might PostgreSQL choose a sequential scan on a 30-row table even after you create a useful index?

Do not turn the page until you can answer in your own words.

DE03-14 - Views and reusable database logic

What you will be able to do

Create a view as a reusable named query while keeping its grain and ownership explicit.

Why this matters in real work

Teams repeatedly need the same cleaned/reporting logic. A view can expose a stable interface without copying a long SELECT into every report.

Picture it this way

A view is like a saved window onto existing tables. It normally stores the query definition, not a second independent copy of every source row.

Start from zero

- **CREATE VIEW name AS SELECT ...** gives a query a reusable database object name.
- Users can query the view like a table for many read operations.
- A view does not automatically solve performance problems.
- Document the view grain: what does one row represent?
- Document ownership and change responsibility because downstream reports may depend on its columns and definitions.

Teacher demonstration

```
CREATE VIEW reporting.completed_order_sales AS
SELECT
    o.order_id,
    o.order_date,
    o.region,
    c.customer_name,
    o.qty * o.unit_price * (1 - o.discount_pct) AS net_sales
FROM sales_orders AS o
JOIN customers AS c
    ON o.customer_id = c.customer_id
WHERE o.status = 'Completed';
```

Read the SQL like English

- The view grain is one row per Completed order.
- The view hides repeated join/calculation logic from downstream users.
- If the business definition of 'Completed' changes, the view owner must manage that change carefully.

Expected check / validation anchor

design field	example
view name	reporting.completed_order_sales
grain	one row per Completed order
source tables	sales_orders + customers
owner concern	definition/version changes affect downstream users

Common beginner traps

- Creating a view without documenting grain.
- Assuming a normal view materializes/stores all output data.
- Changing a widely used view's columns without considering downstream dependencies.

Now you do a similar task

Design a view for latest support ticket per customer. Write the intended grain, source table(s), core query pattern, and one downstream use. You do not need to deploy it until your design is clear.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

What is the most important sentence to write before publishing a reporting view?

Do not turn the page until you can answer in your own words.

DE03-15 - Integrated unseen SQL challenge

What you will be able to do

Combine joins, exclusion logic, windows and validation in a fresh integrated problem.

Why this matters in real work

Real work does not announce 'this is the window-function question'. You need to translate a business request into several SQL steps.

Picture it this way

A practical task is like cooking a full meal after learning individual techniques. The challenge is choosing and sequencing the techniques yourself.

Start from zero

- Business request: find the top 2 **Completed, non-returned** orders in each region by net_sales.
- First identify the base order population.
- Then exclude returned orders safely.
- Join customer_name.
- Calculate net_sales at order grain.
- Rank inside each region with a deterministic tie-breaker.
- Finally keep ranks 1 and 2 and validate counts/totals.

Teacher demonstration

```
-- Do NOT copy this as your independent answer.
-- Use it only as a planning skeleton:
WITH base AS (
    -- one row per eligible order
),
ranked AS (
    -- one row per eligible order + rank within region
)
SELECT ...
FROM ranked
WHERE rank_in_region <= 2;
```

Read the SQL like English

- The skeleton intentionally omits the solution.
- Write a grain comment above each CTE before writing its SELECT.
- Validate that returned orders are truly excluded.
- Validate that each region has at most 2 rows.
- In the supplied data, expected top non-returned Completed orders include East O1025/O1021, North O1016/O1012, South O1020/O1028, West O1006/O1010.

Expected check / validation anchor

region	rank 1	rank 2
East	O1025	O1021

region	rank 1	rank 2
North	O1016	O1012
South	O1020	O1028
West	O1006	O1010

Common beginner traps

- Ranking before removing returned orders, then filtering afterward.
- Using NOT IN without thinking about NULL behavior in a different dataset.
- Failing to define tie-breaking order.
- Trusting eight result rows without reconciling eligibility.

Now you do a similar task

Complete the integrated challenge in M03_QUERY_WORKSPACE_TEACHING_RC2.sql with the Learner Book closed. Save your first attempt before opening any review material.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

In a top-2-per-region problem, at what stage should returned orders be excluded: before ranking or after ranking? Why?

Do not turn the page until you can answer in your own words.

DE03-16 - Timed SQL interview set and explanation

What you will be able to do

Solve several SQL questions under time pressure and explain your reasoning like an interview candidate.

Why this matters in real work

Interviews test more than syntax. They test how you clarify grain, break down a problem, validate results, and communicate uncertainty.

Picture it this way

A driving test does not ask whether you have seen a steering wheel. It asks whether you can operate safely under a realistic constraint.

Start from zero

- Use a **45-minute** timer only if you are ready; the timer measures practice, not worth.
- Read all questions first and allocate time.
- For each question, state the target grain before writing SQL.
- If stuck, write a simpler validation query rather than staring at one giant query.
- At the end, spend a few minutes explaining one answer aloud: inputs, grain, logic, validation, and one failure mode.

Teacher demonstration

```
-- Interview set
-- Q1: Which region has the highest Completed net_sales?
-- Q2: Which customers have at least 2 support tickets?
-- Q3: Return the latest support ticket per customer.
-- Q4: Find the top sales_rep by Completed net_sales, excluding NULL sales_rep.
```

Read the SQL like English

- Known validation anchors for the supplied dataset: East has the highest Completed net_sales (12392.50).
- Customers with at least 2 tickets: C001, C003, C005, C010.
- Latest-ticket IDs: C001/T002, C003/T005, C005/T007, C010/T009, C020/T010.
- Top non-NULL sales_rep by Completed net_sales is Rohit (16422.50).

Expected check / validation anchor

question	validation anchor
Q1	East - 12392.50
Q2	C001, C003, C005, C010
Q3	5 customers with tickets
Q4	Rohit - 16422.50

Common beginner traps

- Opening the answer before saving an attempt.
- Writing SQL before stating what one output row should represent.
- Giving a correct number with no validation or explanation.

Now you do a similar task

Complete the timed set closed-book. Then choose your weakest question, explain the failure, review only that concept, and retry a fresh variation.

How to prove it

- State the output grain in one sentence.
- Run at least one separate validation query or manual check.
- If the result is plausible but your validation disagrees, stop and debug instead of moving on.

STOP - check your understanding

If an interviewer asks 'How do you know this result is correct?', what two concrete validation checks could you describe?

Do not turn the page until you can answer in your own words.